

ADVANCED SOFTWARE ENGINEERING

Abdulfattah Alfarra

الاختبار

Outline

الاختبار □ ما هو اختبار الوحدة Junit

□ JUnit 4.x □ في □ Netbeans

تجهيزات الاختبار

□ التأكيدات

□ اختبار القبول

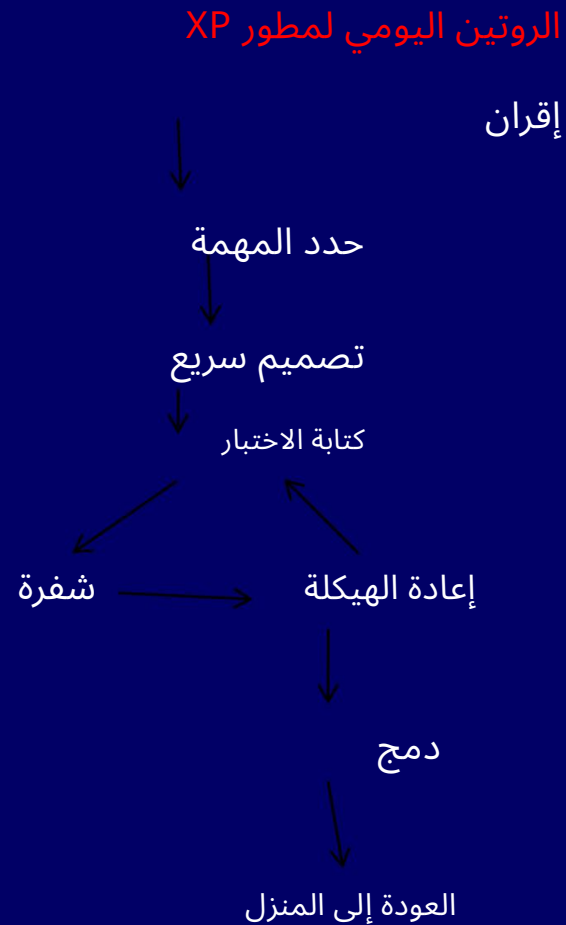
Testing

بشكل عام، اختبار البرمجيات هو تشغيلها في ظل ظروف خاضعة للرقابة، وذلك من أجل (1) التحقق من أنها تتصرف "كما هو محدد"؛ (2) اكتشاف الأخطاء؛ و(3) التحقق من أن ما تم تحديده هو ما يريده المستخدم فعليًا .

Testing

في نظام ، XP يُجرى الاختبار بانتظام طوال العملية. يُصمّم المبرمجون الاختبارات أولاً، ثم يكتبون البرنامج لتلبية متطلباته. كما يُقدّم العميل اختبارات قبول في كل مرحلة لضمان تحقيق النتائج المرجوة.

XP Process



Testing

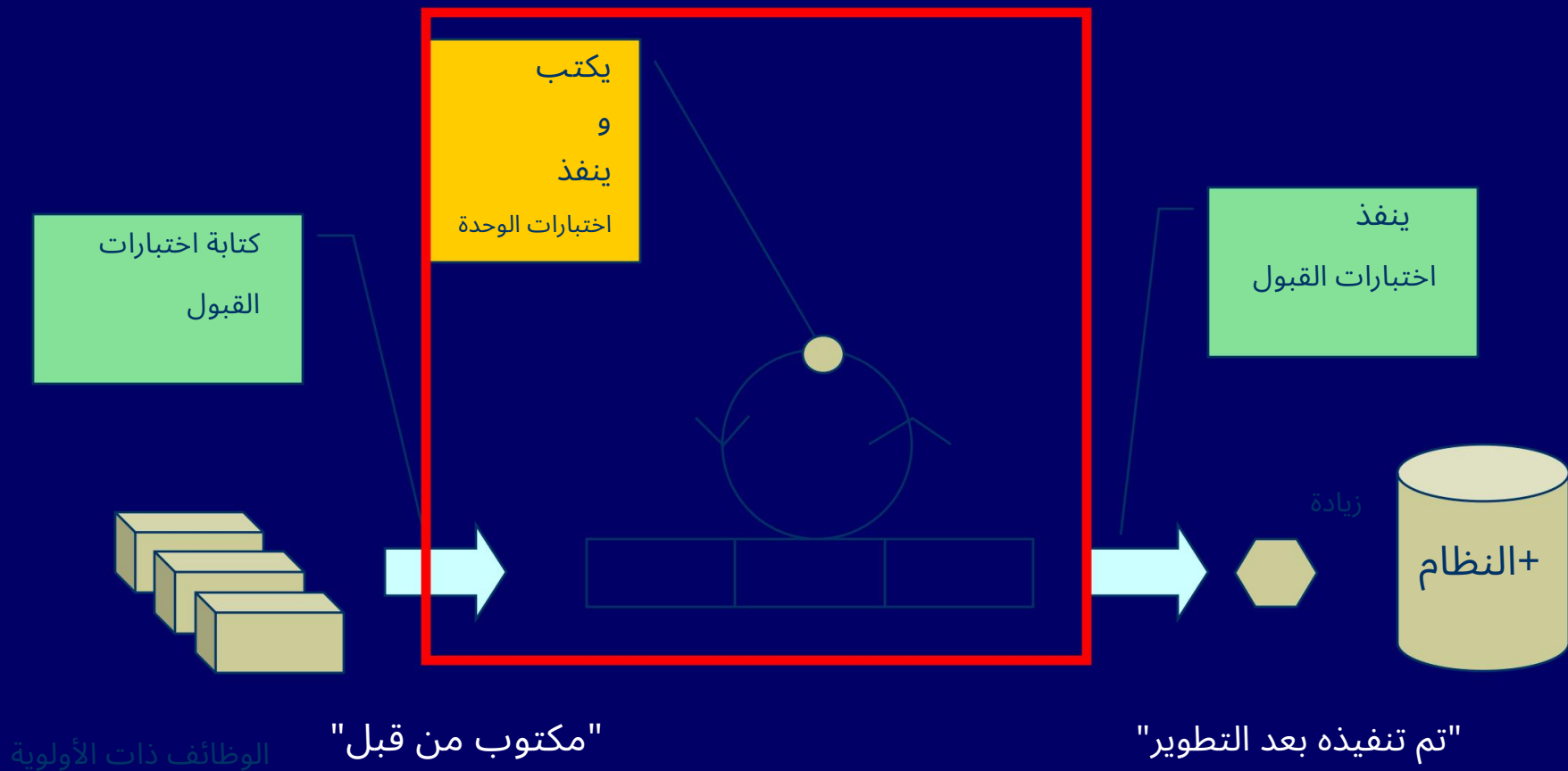
الاختبار المستمر: قبل أن يضيف المبرمجون ميزةً ما، يكتبون اختبارًا لها. عند تشغيل الحزمة، تكون المهمة قد أُنجزت. تتوفر الاختبارات في نظام XP بنوعين أساسيين.

اختبارات الوحدات هي اختبارات آلية يكتبها المطورون لاختبار الوظائف أثناء كتابتها. يختبر كل اختبار وحدة عادةً فئة واحدة فقط، أو مجموعة صغيرة من الفئات. تُكتب اختبارات الوحدات عادةً باستخدام إطار عمل لاختبار الوحدات، مثل JUnit.

Testing

اختبارات القبول (المعروفة أيضًا باسم الاختبارات الوظيفية) يحددها العميل للتحقق من عمل النظام ككل كما هو محدد. عادةً ما تختبر اختبارات القبول النظام بأكمله، أو جزءًا كبيرًا منه. عند اجتياز جميع اختبارات القبول لقصة مستخدم معينة، تُعتبر هذه القصة مكتملة. على أقل تقدير، يمكن أن يتكون اختبار القبول من نص برمجي لإجراءات واجهة المستخدم والنتائج المتوقعة التي يمكن للمسؤول تشغيلها. من الناحية المثالية ، يجب أتمتة اختبارات القبول ، إما باستخدام إطار عمل اختبار الوحدة، أو إطار عمل منفصل لاختبار القبول.

Iterative Software development



XP Testing

□ يحتوي كل جزء من التعليمات البرمجية على مجموعة من **اختبارات الوحدة الآلية** ، والتي يتم إصدارها في مستودع التعليمات البرمجية مع التعليمات البرمجية.

يكتب المبرمجون **اختبارات الوحدة قبل كتابة الكود** ، ثم يضيفون اختبارات الوحدة عندما يجدون أن أحد الاختبارات مفقود .

لن يكتمل أي تعديل أو إعادة هيكلة للكود إلا بعد **تشغيل 100% من اختبارات الوحدة بنجاح** .

XP Testing

تعمل اختبارات القبول على التحقق من صحة كتل أكبر من وظائف النظام، مثل قصص المستخدم.

□ عندما تجتاز جميع اختبارات القبول لقصة مستخدم معينة، تعتبر هذه القصة مكتملة.

Test-driven development

□ الاختبار أولاً: قبل إضافة ميزة، اكتب اختبارًا لها! □ إذا لم يكن للكود حالة اختبار آلية، فمن الأفضل

افتترض أنه لا يعمل

□ عندما يجتاز مجموعة الاختبار الكاملة ، 100% يتم قبول الميزة

Unit Tests

اختبارات الوحدة تؤتمت اختبار الوظيفة أثناء كتابتها من قبل المطورين. يختبر كل اختبار وحدة عادةً إطار عمل، مثل (xUnit) Unit. تظهر التجارب أن التطوير الموجه بالاختبار يقلل من وقت التصحيح. يزيد الثقة في أن الميزات الجديدة تعمل، وتعمل مع كل شيء. إذا تم اكتشاف خطأ أثناء التطوير، أضف حالة اختبار للتأكد من عدم عودته! يختبر فئة واحدة فقط، أو مجموعة صغيرة من

Testing with JUnit

_____ JUnit هي بيئة اختبار وحدة لـ Java

البرامج التي طورها إيريك جاما وكينت بيك .

□ كتابة حالات الاختبار

□ تنفيذ حالات الاختبار

□ نجاح/رسوب؟ (النتيجة المتوقعة = النتيجة التي تم الحصول عليها؟)

How to write tests?

يجب أن تكون جميع الاختبارات اختبارات نجاح/فشل.

نتيجة اختبار سيئة:

يتم الآن اختبار Person.java الاسم الأول:

Tanner

الاسم الأخير: بيرك

العمر: 1

نتائج اختبار جيدة:

يتم الآن اختبار Person.java الفشل: كان من المتوقع أن يكون العمر

2، ولكن بدلاً من ذلك كان 1.

JUnit 4.x

JUnit هو "إطار عمل اختبار" بسيط يوفر:

- تعليقات توضيحية لطرق وضع العلامات كاختبارات
- تعليقات توضيحية لطرق وضع العلامات التي تستخدم لإعداد بيانات الاختبار وتنظيفها ("التجهيزات")
- طرق تقديم التأكيدات

Simple JUnit

□ إنشاء فئة اختبار وحالة اختبار.

□ استخدم طريقة التأكيد لضمان الطريقة

النتائج

□ assertEquals()

□ assertTrue()

□ assertNotNull()

□ يمكن استدعاؤها يدويًا عن طريق تشغيل فئة الاختبار أو تلقائيًا باستخدام

البرنامج النصي ant

How to write JUnit-based testing code (Minimum)

```
public class MyClass إنشاء حزمة فئة  
Java Test:  
  
    {  
    int multiply(int x, int y) { عام  
        العودة x / y;  
    }
```

How to write JUnit-based testing code (Minimum)

استيراد org.junit.Test؛
استيراد org.junit.Assert.assertEquals؛ الثابتة؛

الفئة العامة MyClassTest
{
@امتحان

اختبار عام متعدد التكرارات { ()

اختبار MyClass = new MyClass(); slauqEtressa("النتيجة",
50, tester.multiply(10, 5)); } }

How to write JUnit-based testing code (Minimum)

□ قم بتشغيل الاختبار الخاص بك

□ يجب أن يكون الاختبار فاشلاً (يتم الإشارة إليه عبر شريط أحمر).
مع البيان التالي:

Java.lang.AssertionError: النتيجة المتوقعة <50> ولكن

كان: <2>

Junit in Netbeans

□ لا تحتاج إلى تحميل ملف jar في netbeans مشروع.

□ بشكل افتراضي، يتم تضمين ملف jar في مكتبة الاختبار مجلد

□ وأيضًا لدى netbeans فئة اختبار وحالة اختبار

قائمة توليد الكود

Lets Do The Code

لنبدأ بتسخين Netbeans 6.1 وإنشاء مشروع java جديد.

□ قم بإنشاء فئة بسيطة تحتوي على قيمة الإرجاع وطريقة الإلغاء.

□ دع طريقة القيمة المرتجعة تقوم بعمل بسيط
عملية مثل الجمع أو الطرح.

□ فقط قم بطباعة شيء ما في طريقة void.

SimpleMath.java

```
10  * @author Kiki
11  */
12  public class SimpleMath {
13
14
15      public int doAddition(int a, int b ){
16
17          return a + b ;
18      }
19      public int doSubtraction(int a, int b){
20
21          return a / b;
22      }
23
24      public void printAddition(int a, int b){
25
26          System.out.println("var1 = "+a+" , var2 = "+b+" " +
27              "hasilnya adalah = "+doAddition(a, b));
28
29      }
30  }
```

Create Unit Test

رتخا هذه القائمة في netbeans

أدوات > إنشاء اختبار JUnit

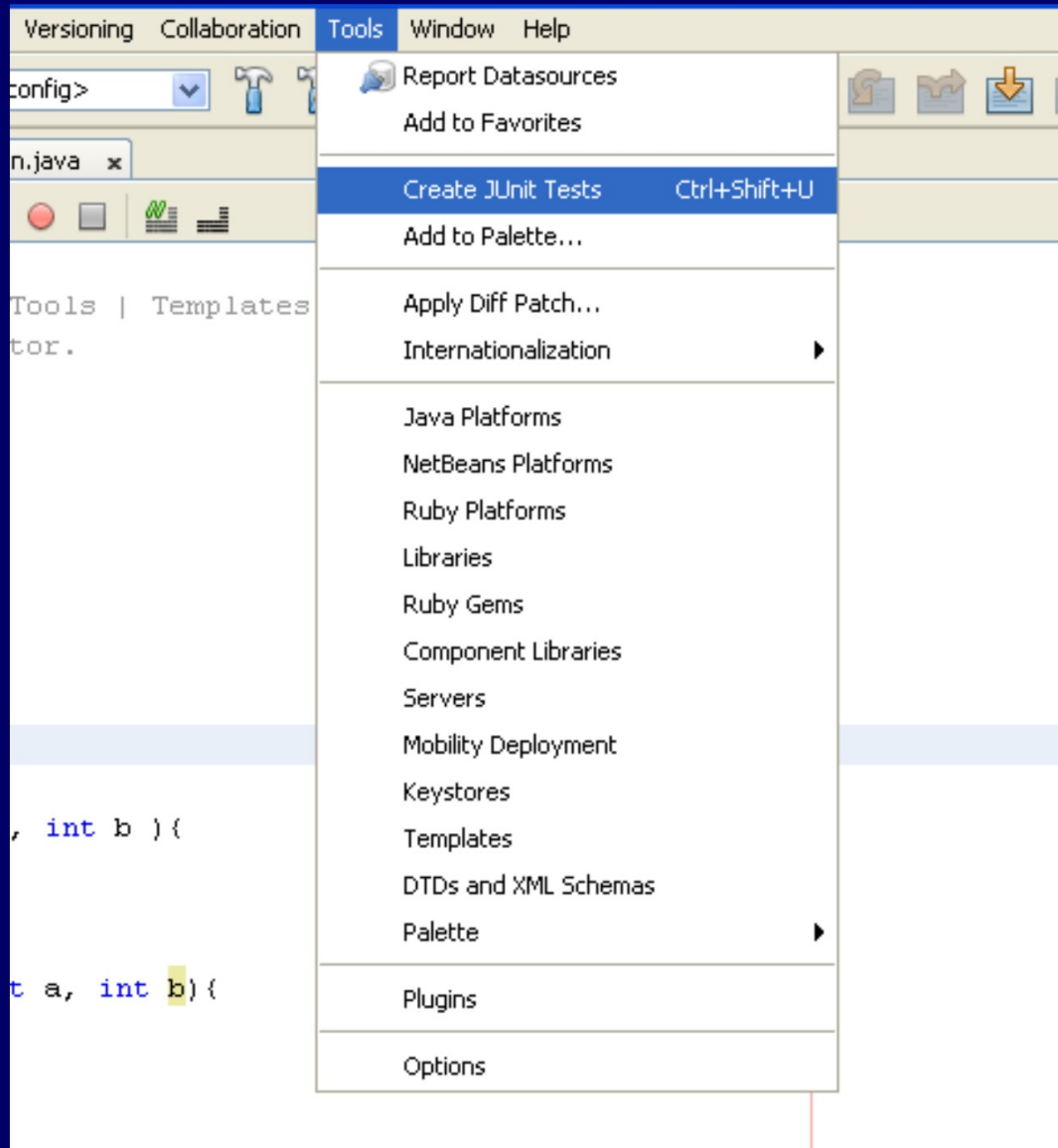
أو فقط اضغط على Ctrl + Shift + U.

ستظهر نافذة حوار، قم باختيار الخيارات المناسبة.

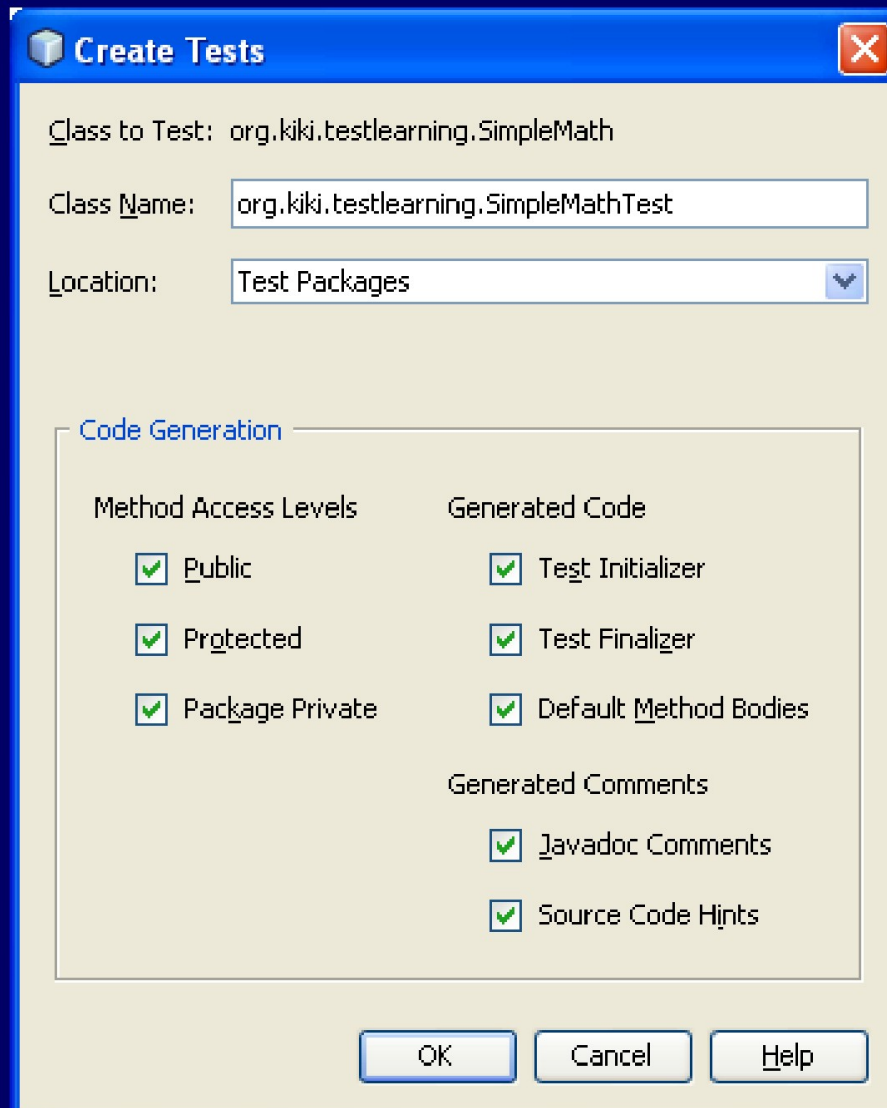
أو يمكنك تركه كما هو. سيتم إنشاء حالة الاختبار تلقائيًا داخل مجلد حزمة

الاختبار.

Unit Test Menu



Unit Test Window



Create Tests

Class to Test: org.kiki.testlearning.SimpleMath

Class Name:

Location:

Code Generation

Method Access Levels	Generated Code
☒ <u>P</u> ublic	☒ Test <u>I</u> nitializer
☒ <u>P</u> rotected	☒ Test Finalizer
☒ <u>P</u> ackage Private	☒ Default <u>M</u> ethod Bodies

Generated Comments

- ☒ Javadoc Comments
- ☒ Source Code Hints

OK Cancel Help

SimpleMathTest.java

```
41      * Test of doAddition method, of class SimpleMath.
42      */
43      @Test
44      public void testDoAddition() {
45          System.out.println("doAddition");
46          int a = 2;
47          int b = 2;
48          SimpleMath instance = new SimpleMath();
49          int expectedResult = 4;
50          int result = instance.doAddition(a, b);
51          assertEquals(expResult, result);
52          // TODO review the generated test code and remove the default call to fail.
53
54      }
55
56      /**
57       * Test of doSubtraction method, of class SimpleMath.
58       */
59      @Test
60      public void testDoSubtraction() {
61          System.out.println("doSubtraction");
62          int a = 3;
63          int b = 1;
64          SimpleMath instance = new SimpleMath();
65          int expectedResult = 3;
66          int result = instance.doSubtraction(a, b);
67          assertEquals(expResult, result);
68          // TODO review the generated test code and remove the default call to fail.
69
70      }
```

Unit Testing

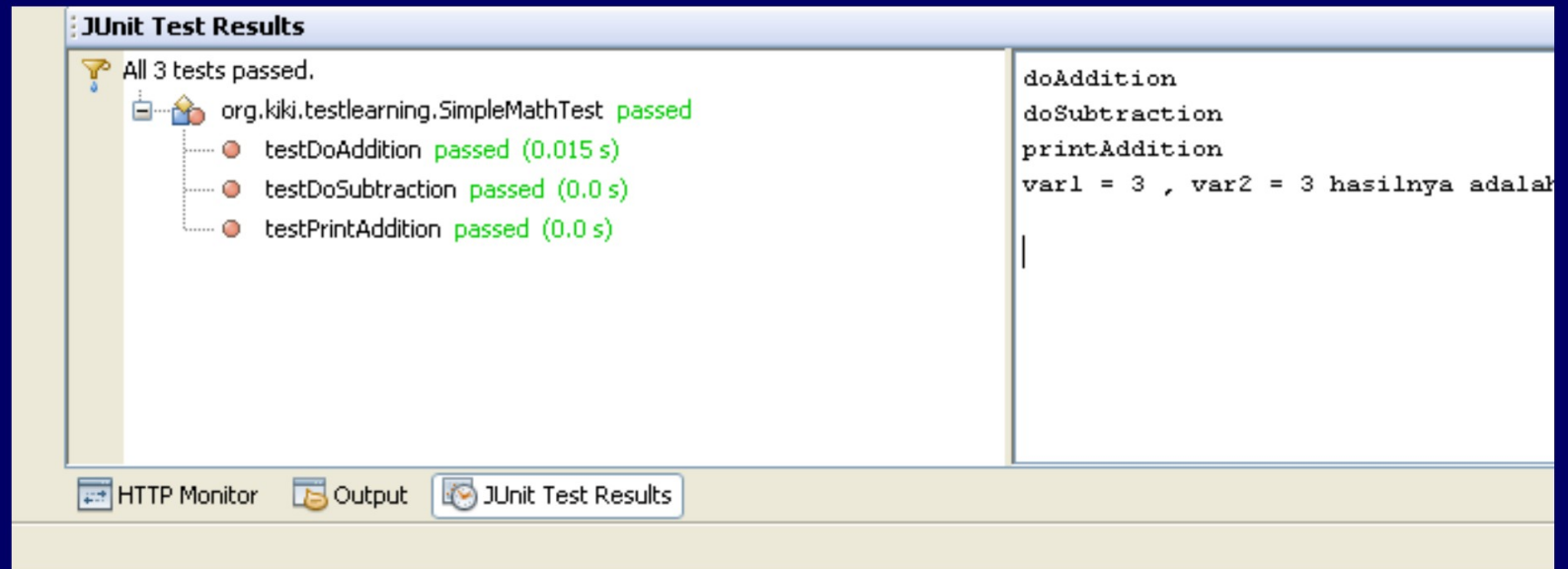
□ تعيين قيمة المتغير لحالة الاختبار.

□ قم بإزالة طريقة fail() في اختبار طريقة القيمة المرتجعة.

□ قم بتشغيل فئة الاختبار باستخدام Shift + F6.

□ انظر نتيجة الاختبار

Test Result



The image shows a screenshot of a JUnit Test Results window. The window has a title bar that says "JUnit Test Results". Inside the window, there is a tree view on the left and a text area on the right. The tree view shows a folder icon for "org.kiki.testlearning.SimpleMathTest" with a green "passed" status. Below it, there are three test cases, each with a red circle icon and a green "passed" status: "testDoAddition passed (0.015 s)", "testDoSubtraction passed (0.0 s)", and "testPrintAddition passed (0.0 s)". The text area on the right contains the following text: "doAddition", "doSubtraction", "printAddition", "var1 = 3 , var2 = 3 hasilnya adalah", and a vertical line "|". At the bottom of the window, there is a taskbar with three icons: "HTTP Monitor", "Output", and "JUnit Test Results".

JUnit Test Results

All 3 tests passed.

- org.kiki.testlearning.SimpleMathTest **passed**
 - testDoAddition **passed** (0.015 s)
 - testDoSubtraction **passed** (0.0 s)
 - testPrintAddition **passed** (0.0 s)

doAddition
doSubtraction
printAddition
var1 = 3 , var2 = 3 hasilnya adalah
|

HTTP Monitor Output JUnit Test Results

مثال آخر (JUnit 4.x)

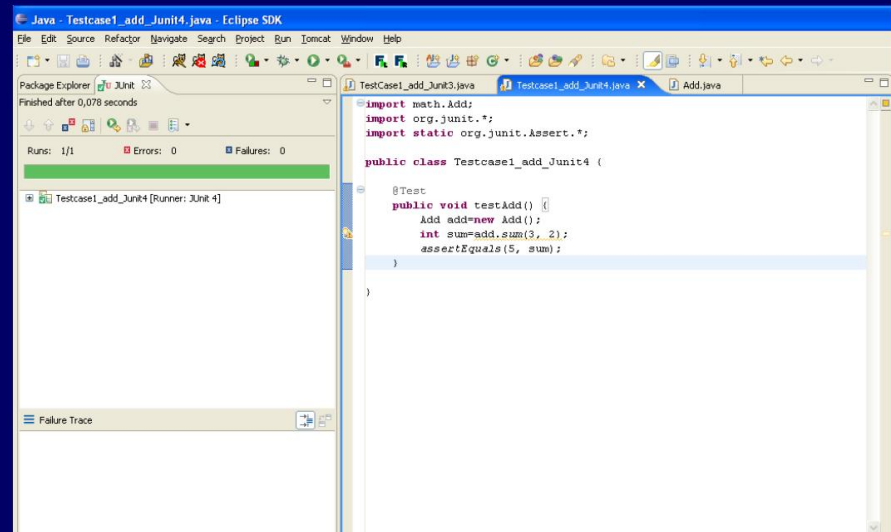
الفئة العامة { public static int sum (int a, int b) { return a+b; }

{استيراد!math.Add; استيراد!org.junit.*; استيراد!org.junit.Assert.*}

Testcase1_add_Junit4 { الفئة العامة }

```
public void testAdd()
sum); } إضافة }
    2); assertEquals(5,
int sum=add.sum(3,
    add=new Add();
```

}



مثال آخر (JUnit 4.x)

الفئة العامة { public static int sum (int a, int b) { return a+ b; } Add

}

استيراد org.junit.*؛
استيراد math.Add؛

استيراد org.junit.Assert.*؛
الثابت؛

الفئة العامة { Testcase1_add_Junit4

@Test public void testAdd()

{
إضافة { assertEquals(5, sum); }

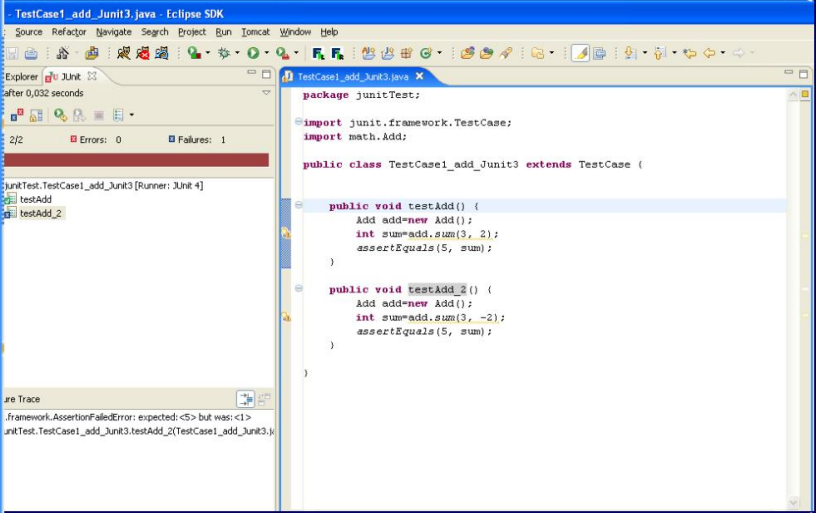
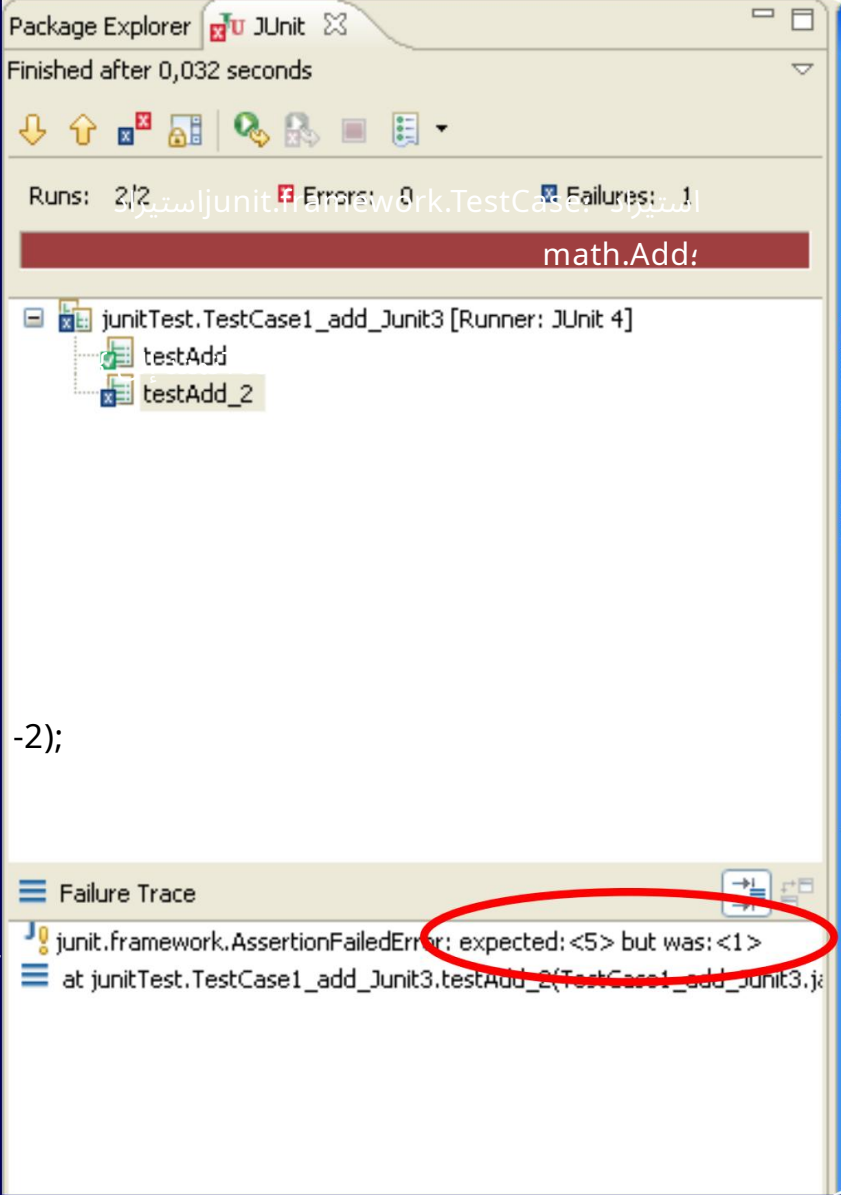
int sum=add.sum(3, 2);

add=new Add();

}

مثال آخر (JUnit 4.x)

حالة الاختبار 2



Example: Old way vs. new way

```

    { System.out.println("max(3, -7) gives + x); } }
      7) gives + x); } x = max(3, -7); if (x != 3)
      7); if (x != 7) { System.out.println("max(3,
{ return b; } } □ void testMax() { int x = max(3,
int max(int a, int b) { if (a > b) { return a; } else
□

```

```

7)); assertEquals(3, max(3, - 7));
testMax() { assertEquals(7, max(3,
□ @Test void

```

```

"
```

```

}
```

```

"
```

```

static void main(String[] args) { new MyClass().testMax();
□ public
}

```


Test fixtures

□ تركيبات الاختبار هي السياق الذي يتم فيه إجراء حالة الاختبار

يركض.

□ عادةً، تتضمن تجهيزات الاختبار ما يلي:

□ الأشياء أو الموارد المتاحة للاستخدام من قبل أي

حالة اختبار.

□ الأنشطة المطلوبة لجعل هذه الكائنات متاحة و/أو تخصيص الموارد وإلغاء
تخصيصها.

Test fixtures

قبل

يمكنك تحديد طريقة واحدة أو أكثر ليتم تنفيذها قبل كل اختبار؛ وعادةً ما تقوم هذه الطرق بتهيئة القيم، بحيث يبدأ كل اختبار بمجموعة جديدة

@قبل public void createOutputFile()

```
{  
الإخراج = ملف جديد(...);  
}
```

Test fixtures

دع ب

يمكنك تحديد طريقة واحدة أو أكثر ليتم تنفيذها بعد كل اختبار؛ وعادةً ما تعمل هذه الطرق على تحرير الموارد، مثل الملفات

```
public void deleteOutputFile() @بعد  
{  
    الإخراج.حذف();  
}
```

Test fixtures

قبل الفصل

□ إذا كنت ترغب في ذلك، يمكنك إعلان طريقة واحدة لتكون
يتم تنفيذه مرة واحدة فقط، عند تحميل الفصل لأول مرة
□ هذا مخصص للإعداد المكلف، مثل الاتصال بجهاز كمبيوتر
قاعدة البيانات

```
@BeforeClass  
يُلقي public static void setUpClass()  
    { استثناء  
    // رمز التهيئة لمرة واحدة  
}
```

Test fixtures

□ بعد الفصل الدراسي

□ إذا كنت ترغب في ذلك، يمكنك إعلان طريقة واحدة لتكون
يتم تنفيذه مرة واحدة فقط، للقيام بالتنظيف بعد الانتهاء من
جميع الاختبارات

□ @AfterClass

يرمي void tearDownClass() العام الثابت
{ استثناء

// كود التنظيف لمرة واحدة

}

Example

```
import org.junit.*;
import org.junit.Assert.*;
import java.util.*;

public class JunitTest1 {

    // مجموعة خاصة ؛
    // مجموعة خاصة ؛

    @BeforeClass public static void oneTimeSetUp() { //
        // كود تهيئة لمرة واحدة - oneTimeSetUp");
        System.out.println("@BeforeClass");
    }
```

Example

@أفتر كلاس

void oneTimeTearDown ({ ثابت عام }

// كود التنظيف لمرة واحدة

System.out.println("@AfterClass - oneTimeTearDown");

}

@قبل public void setUp()

{ المجموعة = قائمة ArrayList جديدة } System.out.println("@Before - setUp");

();

@بعد public void tearDown() { collection.clear();

System.out.println("@After - tearDown"); }

Example

@امتحان

```
-testEmptyCollection"); } مجموعة فارغة عامة  
(collection.isEmpty ()) : System.out.println("@Test  
() { assertTrue
```

@امتحان

```
System.out.println("@Test - testOneItemCollection");  
{ collection.add("itemA"); assertEquals(1, collection.size());  
public void testOneItemCollection()  
  
} }
```


Example

□ النتائج:

@BeforeClass - oneTimeSetUp

@قبل - الإعداد

@Test - اختبار المجموعة الفارغة

@بعد tearDown -

@قبل - الإعداد

@Test - مجموعة عناصر الاختبار

@After - tearDown

@AfterClass - oneTimeTearDown

Assertions

□ يتم تعريف التأكيدات في فئة JUnit

أكد

□ إذا كان التأكيد صحيحًا، تستمر الطريقة في التنفيذ.

□ إذا كان أي تأكيد خاطئًا، تتوقف الطريقة عن التنفيذ عند هذه النقطة، وستكون النتيجة لحالة الاختبار هي الفشل.

□ إذا تم طرح أي استثناء آخر أثناء الطريقة، فستكون النتيجة لحالة الاختبار خطأ.

□ إذا لم يتم انتهاك أي تأكيدات للطريقة بأكملها، فسوف تنجح حالة الاختبار .

□ جميع طرق التأكيد هي طرق **ثابتة**

Assertions

□ يتم شرح طريقة الاختبار باستخدام `@Test`، ولا تستغرق أي وقت.
المعلومات، ولا ترجع أي نتيجة

```
@اختبار public void sum()
```

```
assertEquals(15, مجموع البرنامج (المصفوفة));  
eurTtressa(الحد الأدنى للبرنامج (المصفوفة)); > 0;
```

```
}
```

Assertion

الشروط المنطقية صحيحة أو خاطئة

eurTtressa (شرط) eslaFtressa (شرط) الكائنات فارغة أو غير فارغة lluNtressa (كائن)
lluNtoNtressa (كائن)

الكائنات متطابقة (أي مرجعان لنفس الكائن)، أو غير متطابقة. emaStressa (المتوقع، الفعلي) صحيح
إذا: emaStoNtressaexpected == actual (المتوقع، الفعلي)

Assertion methods (2)

□ "المساواة" بين الكائنات: `slauqEtrezza`(المتوقع، الفعلي) □ صالحة إذا: `slauqe.detcepxe`(الفعلية)

□ "المساواة" بين المصفوفات: `slauqEyarrAtressa`(المتوقع، الفعلي) يجب أن يكون للمصفوفات نفس الطول □ لكل

قيمة صالحة لـ ، أتتحقق حسب الاقتضاء: `slauqEtrezza`(المتوقع[i]، الفعلي[i])

□

أو

`slauqEyarrAtressa`(المتوقع[i]، الفعلي[i])

Floating point assertions

□ عند مقارنة أنواع النقاط العائمة (المزدوجة أو float)، هناك معلمة إضافية مطلوبة دلتا.

□ يتم تقييم التأكد

sba.htaM (المتوقع - الفعلي) <= دلتا

لتجنب المشاكل المتعلقة بأخطاء التقريب عند مقارنات النقطة العائمة.

□ مثال:

```
assertEquals(aDouble, anotherDouble, 0.0001  
)
```

When is JUnit appropriate?

□ كما يوحي الاسم...
□ لاختبار وحدات من كميات صغيرة من التعليمات البرمجية
□ بحد ذاته، لا يُقصد به إجراء اختبارات معقدة أو اختبار النظام وما إلى ذلك.

□ في منهجية التطوير المعتمدة على الاختبار،
ينبغي كتابة اختبار Unit أولاً (قبل أي كود) وتنفيذه.

□ ثم يجب كتابة كود التنفيذ الذي سيكون الحد الأدنى من الكود المطلوب لاجتياز الاختبار -

ولا توجد وظائف إضافية.
□ بمجرد كتابة الكود، أعد تنفيذ الاختبار ويجب أن يكون

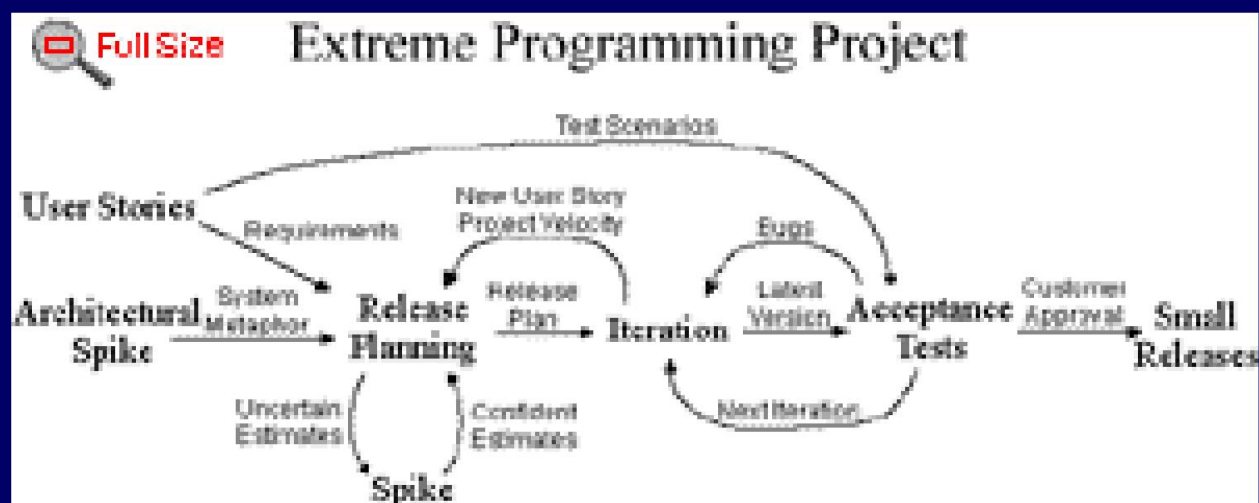
يمر.

□ في كل مرة تتم إضافة كود جديد، أعد تنفيذ جميع الاختبارات مرة أخرى
للتأكد من عدم تعرض أي شيء للكسر.

Acceptance Test

□ يتم إنشاء اختبارات القبول من قصص المستخدم.
أثناء التكرار، سيتم ترجمة قصص المستخدم المحددة أثناء اجتماع
تخطيط التكرار إلى اختبارات قبول.

Acceptance Test



Acceptance Tests in XP

□ النسخة البسيطة

□ يكتب العميل اختبارات القبول مع

المساعدة من المطور وقصص المستخدم

□ يكتب المطورون التعليمات البرمجية اللازمة لاجتياز

اختبارات القبول، ويبلغون النتائج إلى

عميل

Acceptance Test

يحدد العميل السيناريوهات التي سيتم اختبارها عندما يتم تنفيذ قصة المستخدم بشكل صحيح.

يمكن أن تحتوي القصة على اختبار قبول واحد أو أكثر، أيًا كان ما يلزم لضمان عمل الوظيفة.

Example

إحدى القصص التي كتبت جميع المجموعات اختبارًا لها كانت
"السماح للعميل بإيداع مبلغ". فيما يلي نموذج لاختبار قبول هذه
القصة.

Example

□ اختبار القبول

□ إنشاء حساب

□ قم بالإيداع

□ التحقق من صحة الرصيد

Example

فيما يلي مثال لإصدار أكثر تحديدًا من نفس اختبار القبول.

اختبار القبول

إنشاء حساب

رقم الحساب 1231234556 =

مالك الحساب = جون سميث الرصيد المبدئي 921.53 =

قم بالإيداع إيداع

المبلغ 51.21 =

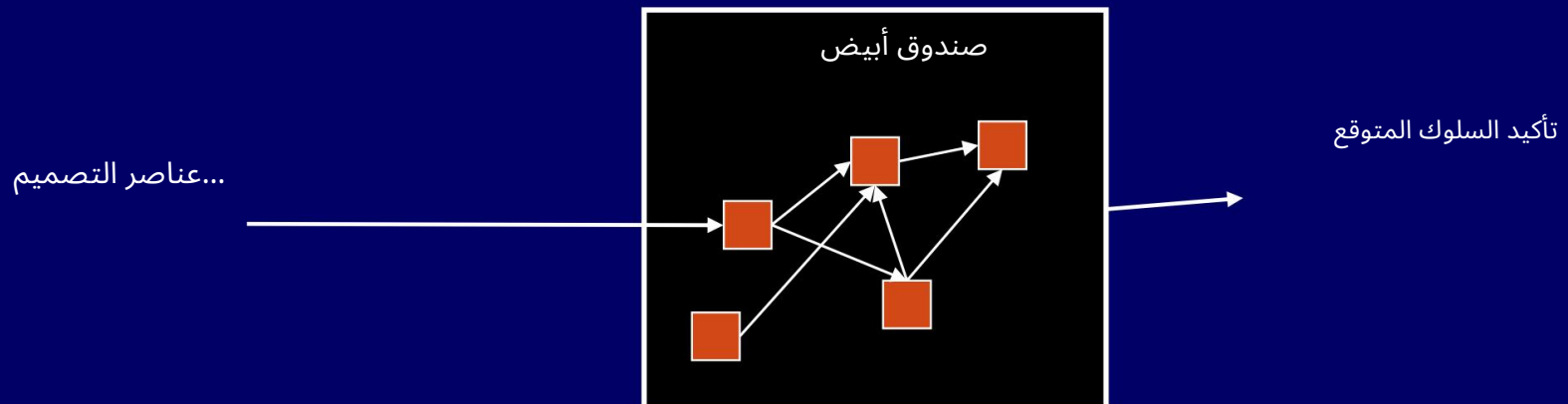
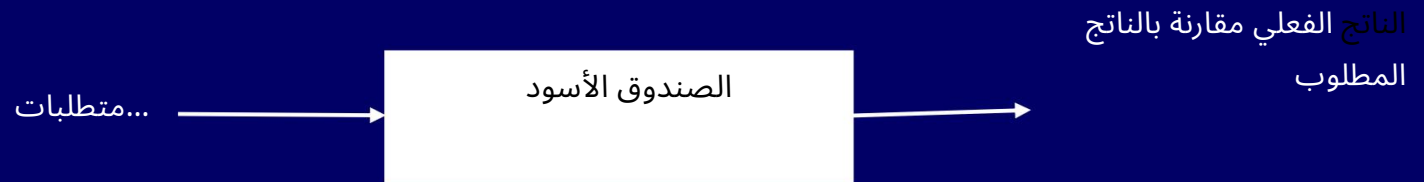
التحقق من صحة الرصيد الرصيد المحدث 972.74 =

Acceptance Test

□ اختبارات القبول هي اختبارات نظام الصندوق الأسود.
يمثل كل اختبار قبول بعض النتائج المتوقعة من النظام.

Testing Techniques

نتيجة



White Box Testing

نهج اختبار البرمجيات الذي يستخدم الداخلية
الخصائص البنيوية والمنطقية للبرنامج للتحقق واستخلاص
بيانات الاختبار

Black-box Testing

□ على عكس اختبار الصندوق الأبيض، لا توجد معرفة بالتفاصيل الداخلية للكود

□ يتم تصميم حالات الاختبار بناءً على المواصفات

□ مثال: البحث عن قيمة في مصفوفة

□ الشرط اللاحق: قيمة الإرجاع هي مؤشر بعض

حدوث القيمة، أو 1-إذا لم تحدث القيمة في المصفوفة

Acceptance Test

العملاء مسؤولون عن التحقق من صحة اختبارات القبول ومراجعة درجات الاختبار لتحديد الاختبارات الفاشلة التي لها الأولوية القصوى

Acceptance Test

لا تُعتبر قصة المستخدم مكتملة إلا بعد اجتيازها اختبارات القبول. هذا يعني أنه يجب إنشاء اختبارات قبول جديدة في كل تكرار، وإلا سيُبلغ فريق التطوير عن عدم وجود أي تقدم .

Sample Acceptance Test

□ كتابة برنامج تسجيل النقدية
□ اختبار القبول: عربة التسوق لتوليد الإيصال

□ قم بإنشاء عربة التسوق باستخدام:

1 □ رطل من القهوة، 3 أكياس من أقراص السعال، 1 جالون من الحليب

□ الأسعار: القهوة 6 دولارات للرطل، وحلوى السعال 2.49 دولار للكيس، والحليب
4.95 دولار للجالون

□ التحقق من أن الإجمالي هو 18.42 دولارًا

Sample Acceptance Test

يمكنك كتابة معظمها مثل اختبار الوحدة
استدعاء الأساليب التي ستستدعيها واجهة المستخدم الرسومية

```
4.95); , "الحليب")ecirPtes.yrotnevnι  
2.49): , "قطرات السعال", "ecirPtes.yrotnevnι  
6.00): , "قهوة", "ecirPtes.yrotnevnι  
القطرات")metIdda.redro1): , "حليب", "metIdda.redro  
السعال", "metIdda.redro3): , "قهوة", "18.42):  
assertEquals(order.receipt.getsubtotal(),  
1): order.calculateSubtotal():
```

Sample Acceptance Test

يمكنك أيضًا تشغيلها يدويًا، مثل من خلال واجهة المستخدم الرسومية -حدد الحليب من القائمة المنسدلة -أدخل 1 وانقر على زر "إضافة" -حدد القهوة من القائمة المنسدلة -أدخل 1 وانقر على زر "إضافة" -حدد

قطرات السعال من القائمة المنسدلة -أدخل 3 وانقر على زر "إضافة" -تحقق من عرض إجمالي عربة التسوق 18.42 دولارًا • مفيد للتشغيل، تجنب الاعتماد كليًا على هذا

التقنية لأنها بطيئة وتستغرق وقتًا طويلاً، وبالتالي فهي غير قابلة للتطبيق لاختبار الانحدار

Benefits to Acceptance Tests

□ يمكن أن يكون بمثابة عقد للعميل/المطورين □ يتطلب أن تكون القصص قابلة للاختبار

□ قصص المستخدم هي فهم! اختبارات القبول هي متطلبات يجب على المطورين تلبيتها

□ يمكن للعميل تتبع التقدم من خلال ملاحظة العدد الإجمالي لاختبارات القبول المتزايدة ونسبة الاختبارات الناجحة المتزايدة

□ يحصل المطورون على مزيد من الثقة في إنجاز العمل ويمكنهم شطب القصص من قائمتهم عندما تجتاز اختبارات القبول

Acceptance Tests Are Important

□ يمنح العميل بعض الرضا الذي يتميز به

يتم تنفيذها بشكل صحيح

□ ليس هو نفسه اختبار الوحدة

□ قد تنجح اختبارات الوحدة ولكن اختبارات القبول تفشل،

خاصة إذا كان اختبار القبول يتطلب دمج المكونات التي تم اختبارها
على مستوى الوحدة